

Build Prompt: Campus Task App

Copy everything below into Antigravity as your project prompt.

Project Overview

Build a full-stack web application called "**Campus Task App**" — a hyperlocal task marketplace exclusively for VIT students. Students can post small tasks (errands, tutoring, freelancing, etc.) that other verified students can apply for and complete in exchange for payment. The platform is closed — only verified college email addresses can sign up.

Tech stack preference: Use a modern full-stack setup — React (frontend), Node.js/Express or Next.js API routes (backend), PostgreSQL (database), and an auth provider that supports email domain restriction (e.g., Supabase Auth, Clerk, or custom JWT + email OTP). Use whatever stack you judge most maintainable, but justify the choice briefly before scaffolding.

Build this iteratively: scaffold the project structure first, implement one module at a time in the order listed below, and confirm each module works before moving to the next.

1. Authentication

- Sign-up/login restricted to email domains: `@vitstudent.ac.in` and `@vit.ac.in` only. Reject all other domains at signup with a clear error message.
- Use email OTP or magic link verification (no password-only signup).
- Store a `verified` boolean on the user record once email is confirmed.
- No social login, no phone-only login.

2. Database Schema (design this first)

Design tables/models for at minimum:

- `users` — id, username, email, verified, credits, bio, hostel_block, created_at
- `tasks` — id, poster_id, title, description, category, photo_url, budget, payment_amount, deadline, location, people_needed, assignment_mode (`first_come` | `review_select`), status (`open`, `assigned`, `in_progress`, `completed`, `cancelled`), created_at
- `task_applications` — id, task_id, applicant_id, status (`pending`, `accepted`, `rejected`), offer_amount (nullable, for counter-offers), created_at

- `reviews` — id, task_id, reviewer_id, reviewee_id, role (`poster` | `doer`), rating, comment, created_at
- `badges` — id, user_id, badge_type, earned_at
- `messages` — id, task_id, sender_id, content, attachment_url, attachment_type, created_at
- `credit_transactions` — id, user_id, amount, reason, created_at
- `support_tickets` — id, user_id, type (`contact`, `feedback`, `bug`), subject, message, status, created_at

Add indexes on foreign keys and on `tasks.status` / `tasks.category` for browse-page performance.

3. Profile Page

Single page, public to all logged-in users, viewable for any username.

Shared section (top):

- Username, verified badge, credits total, bio

Two-column layout below:

- **Left column — "As Poster":** tasks posted, completion rate, rating, member since, cancellation rate, reviews/comments, hostel block/general area
- **Right column — "As Doer":** tasks completed, completion rate, rating, member since, skill tags, reviews/comments, badges (5/10 tasks completed, 5/10 tasks posted — auto-awarded when thresholds are hit)

Private section (owner-only view): earning history (list of credit transactions with reasons/dates)

Compute completion rate and cancellation rate from the `tasks` / `task_applications` tables, not stored as static fields.

4. Task Creation

Form fields: title, description, photo attachment (limit to 5MB, jpg/png only), category (dropdown: Errands, Second-hand items, Tutoring, Freelancing, Transportation), number of people needed, budget, payment amount to acceptor, deadline (date/time picker), location (text or map pin).

Assignment mode toggle, chosen at creation:

- **First-come, first-served:** first applicant is auto-assigned, task status flips to `assigned` immediately, all other applications auto-rejected.

- **Review & select:** task stays `open` collecting applications until poster manually reviews a list of applicants and picks one (or more, up to `people_needed`) from a preview screen.

Validate: deadline must be in the future, budget must be > 0 , `people_needed` ≥ 1 .

5. Browsing Page

- Search bar (full-text search on title + description)
- Filters: category, budget range, deadline range, date posted range
- Sort options: budget low $\rightarrow$ high, budget high $\rightarrow$ low, quick filters for "above ₹100" / "below ₹100"
- Each task card shows: title, category, budget, number of applicants so far, number of people needed, deadline, distance/location if available
- Default view: open tasks only, sorted by newest first

6. Notifications (Email)

Trigger transactional emails on:

- Task assignment $\rightarrow$ poster receives "Your task has been assigned to [username]"
- Task assignment $\rightarrow$ doer receives "You have been assigned to [task title]"

Use a transactional email service (e.g., Resend, SendGrid, or Nodemailer with SMTP) — pick one and wire it up with environment variables for API keys.

7. Offers

- On any open task, a doer can submit an "offer" with a custom price instead of just applying at the listed budget.
- Offer appears in the poster's inbox as an "offer request" alongside a chat thread starter.
- Poster can accept (which assigns the task at the offered price), reject, or counter via chat.

8. Chat Page

- Inbox: list of conversations, each tied to a specific task (show task title + other party's username + last message preview)
- Within a thread: support file sharing for PDF, DOC/DOCX, and photos, capped at a reasonable size limit (10MB for docs, 5MB for images)

- Quick reply buttons: "On my way", "Running late by 10 mins" — insert as a normal message when tapped
- When a task's status changes to `completed`, lock the corresponding chat thread (read-only, no new messages)

9. Account Settings

- Change username (check uniqueness)
- Change password (if using password-based auth) or re-verify email (if using OTP/magic link only)
- Logout

10. Support

- "Contact us" form: subject + message, stored as a `support_tickets` row (type: `contact`) and also emailed to an admin address from env config
- Separate "Submit feedback" and "Report a bug" forms, same underlying ticket table with different `type` values
- Simple admin view (can be minimal/internal) to list and mark tickets as resolved

11. Leaderboard

- Page ranking users by total credits, descending
- Show username, verified badge, credit total, and rank position
- Paginate or limit to top 50

Credit System (implement exactly as follows)

Event	Credit Change
Task completed as doer	+10
Task completed as poster (task you posted gets completed successfully)	+5
Cancel after being assigned (as doer)	-10
Cancel after assigning someone (as poster)	-5
Task flagged in a dispute and resolved against you	-15

Log every change in `credit_transactions` with a `reason` string so the private earning history page has something to display. (Adjust these numbers if you have different preferences — this is a placeholder you can tune later.)

Cancellation & Dispute Handling

- Either party can cancel an assigned task before completion; apply the credit penalty above and revert task status to `open` (if poster cancels) or back to accepting applicants (if doer cancels).
- After a doer marks a task "delivered," the poster has a window (e.g., 48 hours) to confirm completion or raise a dispute. Disputes create a `support_tickets` entry (type: `dispute`) for manual admin resolution — no automated resolution logic needed for v1.

Content Policy (enforce at task-creation validation)

Block task creation if title/description matches disallowed patterns: writing assignments/essays for someone to submit as their own, taking exams/tests on someone's behalf, or any phrasing implying academic work submitted under someone else's name. Allow tutoring/explanation/proofreading-type language. Use a simple keyword/phrase blocklist for v1 (can upgrade to a smarter classifier later) and reject with a clear message explaining why.

Build Instructions for Antigravity

1. Scaffold the project (frontend + backend + database schema/migrations) and confirm it runs locally before adding features.
2. Implement modules in this order: Auth → Database schema → Profile page → Task creation → Browsing page → Offers → Chat → Notifications → Account settings → Support → Leaderboard → Credit system wiring.
3. After each module, run/build the app and report what works and what's stubbed.
4. Use environment variables for all secrets (DB connection string, email API key, JWT secret) — never hardcode them.
5. Write basic seed data (a handful of fake users and tasks) so the browsing/profile/leaderboard pages aren't empty during development.
6. Flag any ambiguity back to me rather than guessing silently — especially around the credit values and dispute window, which are placeholders I may want to change.